Retrieval-Augmented Generation (RAG)

1. Información general

Tema: Retrieval-Augmented Generation
Abreviatura: RAG
Nivel: Intermedio–avanzado
Duración sugerida: 2 a 3 horas
Enfoque: Enterprise AI Platforms, LLM applications y sistemas empresariales de conocimiento.

---

2. Objetivos de aprendizaje

Al finalizar esta sesión, serás capaz de:

- Explicar qué es RAG y por qué se utiliza.
- Describir el flujo completo de una arquitectura RAG.
- Diferenciar entre ingestión, recuperación, construcción de contexto y generación.
- Explicar qué son los embeddings y las bases de datos vectoriales.
- Comprender el papel de los chunks, metadata, filtros y rerankers.
- Identificar las principales causas de errores en un sistema RAG.
- Explicar cómo evaluar la calidad de un sistema RAG.
- Diseñar una arquitectura RAG para un entorno empresarial.
- Identificar riesgos relacionados con seguridad, costos, latencia y aislamiento de datos.

---

3. ¿Qué es RAG?

Retrieval-Augmented Generation, o generación aumentada mediante recuperación, es una arquitectura que combina:

1. Un sistema de búsqueda o recuperación de información.
2. Un modelo de lenguaje grande, conocido como LLM.

La idea principal es que el modelo no debe responder únicamente utilizando la información aprendida durante su entrenamiento. En lugar de eso, el sistema recupera información relevante desde fuentes externas y se la proporciona al modelo como contexto antes de generar una respuesta.

En términos simples:

> RAG permite que un LLM consulte información externa y actualizada antes de responder.

Las fuentes externas pueden ser:

- Documentos PDF.
- Manuales.
- Bases de conocimiento.
- Wikis.
- Tickets de soporte.
- Documentación técnica.
- Contratos.
- Políticas internas.
- Bases de datos.
- SharePoint.
- Sistemas empresariales.
- APIs.
- Repositorios de código.

---

4. ¿Por qué necesitamos RAG?

Un LLM tradicional presenta algunas limitaciones:

4.1 Conocimiento limitado

El modelo fue entrenado con información disponible hasta una determinada fecha. Por lo tanto, puede desconocer:

- Nuevas políticas.
- Cambios recientes en productos.
- Documentos internos.
- Información privada de una empresa.
- Nuevos procedimientos.
- Datos operativos en tiempo real.

4.2 Falta de acceso a información privada

Un modelo general no conoce automáticamente los documentos internos de una organización.

Por ejemplo, un LLM público no debería asumir que conoce:

- Los procedimientos internos de Wolters Kluwer.
- Los contratos de una empresa.
- Los incidentes de ServiceNow.
- Las políticas de Recursos Humanos.
- La documentación privada de un cliente.

4.3 Riesgo de alucinaciones

Cuando el modelo no conoce la respuesta, puede producir una respuesta aparentemente convincente pero incorrecta.

RAG ayuda a reducir este riesgo al proporcionar información relevante y verificable.

Sin embargo:

> RAG no elimina automáticamente las alucinaciones. Solamente proporciona al modelo información adicional que puede utilizar para responder.

4.4 Necesidad de citar fuentes

En aplicaciones empresariales es importante responder:

- ¿De dónde salió esta información?
- ¿Qué documento la respalda?
- ¿Qué versión del documento se utilizó?
- ¿Qué fragmento fue utilizado para construir la respuesta?

RAG facilita la inclusión de referencias y citas, siempre que el sistema conserve correctamente la relación entre los fragmentos recuperados y sus documentos originales.

---

5. RAG no es entrenamiento del modelo

Es importante distinguir RAG de fine-tuning.

Característica	RAG	Fine-tuning
Modifica los pesos del modelo	No	Sí
Utiliza documentos externos en tiempo de consulta	Sí	No necesariamente
Permite actualizar conocimiento rápidamente	Sí	Requiere nuevo entrenamiento o ajuste

Adecuado para información privada	Sí	Puede serlo, pero requiere controles adicionales
Permite citar documentos	Sí	No de forma natural
Cambia el comportamiento o estilo del modelo	Limitadamente	Sí
Costos principales	Indexación, búsqueda y contexto	Entrenamiento y mantenimiento del modelo

Ejemplo

Si una empresa actualiza su política de vacaciones, con RAG se puede:

1. Cargar el documento actualizado.
2. Procesarlo.
3. Generar nuevos embeddings.
4. Actualizar el índice.
5. Consultar la nueva información.

No es necesario volver a entrenar el LLM.

---

6. Arquitectura general de RAG

Una arquitectura RAG normalmente tiene dos flujos principales:

1. Flujo de ingestión o indexación.
2. Flujo de consulta o generación.

6.1 Flujo de ingestión

Este flujo prepara la información para que pueda ser recuperada posteriormente.

```
None
Documentos y fuentes
        |
        v
Extracción de texto
        |
        v
Limpieza y normalización
```

```
        |
        v
División en chunks
        |
        v
Generación de embeddings
        |
        v
Almacenamiento en índice vectorial
        |
        v
Metadata e información de origen
```

6.2 Flujo de consulta

Este flujo se ejecuta cuando un usuario realiza una pregunta.

```
None
Pregunta del usuario
        |
        v
Procesamiento de la consulta
        |
        v
Generación del embedding de la pregunta
        |
        v
Búsqueda de fragmentos relevantes
        |
        v
Filtros y/o reranking
        |
        v
Construcción del contexto
        |
        v
Prompt para el LLM
        |
        v
```

```
Generación de respuesta
        |
        v
Respuesta con citas y referencias
```

---

7. Componentes principales de RAG

7.1 Data Sources

Son las fuentes originales de información.

Ejemplos:

- SharePoint.
- Confluence.
- Google Drive.
- Bases de datos.
- Archivos PDF.
- Archivos Word.
- Repositorios Git.
- Sistemas de tickets.
- APIs internas.
- Object storage como Amazon S3 o Azure Blob Storage.

En un entorno empresarial, es importante identificar:

- Propietario de los datos.
- Nivel de sensibilidad.
- Permisos de acceso.
- Fecha de actualización.
- Versión del documento.
- Sistema de origen.
- Política de retención.

7.2 Document Ingestion

La ingestión es el proceso de incorporar información al sistema RAG.

Normalmente incluye:

1. Conexión con la fuente.
2. Descarga o lectura del contenido.
3. Extracción del texto.
4. Conversión a un formato común.
5. Limpieza.
6. Enriquecimiento con metadata.
7. División en chunks.
8. Generación de embeddings.
9. Indexación.

Ejemplo

Un documento PDF puede contener:

- Texto.
- Tablas.
- Encabezados.
- Pies de página.
- Imágenes.
- Diagramas.
- Metadatos.

La ingestión debe decidir qué información conservar y cómo representarla.

Consideraciones empresariales

- Procesamiento incremental.
- Detección de documentos modificados.
- Eliminación de documentos eliminados en el origen.
- Control de versiones.
- Reintentos.
- Manejo de errores.
- Idempotencia.
- Trazabilidad.
- Dead-letter queues.
- Validación de documentos.

---

8. Chunking

8.1 ¿Qué es un chunk?

Un chunk es un fragmento de un documento que se almacena y recupera como una unidad independiente.

Por ejemplo, un documento de 50 páginas puede dividirse en fragmentos de aproximadamente 300 a 800 tokens.

La división permite recuperar solamente la parte relevante en lugar de enviar el documento completo al LLM.

8.2 ¿Por qué es importante?

Un chunk demasiado grande puede contener información irrelevante y consumir demasiado contexto.

Un chunk demasiado pequeño puede perder el significado original.

El objetivo es encontrar un equilibrio entre:

- Relevancia.
- Contexto.
- Precisión.
- Costo.
- Latencia.
- Tamaño máximo del contexto del modelo.

8.3 Estrategias de chunking

Chunking por tamaño

Divide el texto cada cierto número de tokens o caracteres.

Ventaja:

- Fácil de implementar.

Desventaja:

- Puede separar ideas relacionadas.

Chunking por párrafos

Utiliza párrafos como unidades de división.

Ventaja:

- Conserva cierta estructura semántica.

Desventaja:

- Los párrafos pueden tener tamaños muy diferentes.

Chunking por encabezados

Divide el documento utilizando títulos y subtítulos.

Ventaja:

- Conserva la estructura lógica del documento.

Desventaja:

- Requiere documentos bien estructurados.

Chunking semántico

Divide el texto según cambios en el significado o tema.

Ventaja:

- Puede conservar mejor las unidades conceptuales.

Desventaja:

- Es más complejo y puede ser más costoso.

Chunking jerárquico

Mantiene una relación entre:

- Documento.
- Sección.
- Subsección.
- Fragmento.

Esto permite recuperar un fragmento específico, pero también incluir información de su sección o documento padre.

8.4 Overlap

El overlap es la cantidad de contenido compartido entre dos chunks consecutivos.

Ejemplo:

```
None
Chunk 1: A B C D E F
Chunk 2: E F G H I J
```

Los elementos `E` y `F` representan el overlap.

El overlap ayuda a evitar que una idea quede dividida entre dos fragmentos sin suficiente contexto.

Sin embargo, demasiado overlap puede:

- Duplicar información.
- Aumentar el tamaño del índice.
- Incrementar costos.
- Generar resultados repetidos.

8.5 No existe un tamaño universal

Una configuración como “500 tokens con 50 tokens de overlap” puede ser un punto de partida, pero no es una regla universal.

El tamaño adecuado depende de:

- Tipo de documento.
- Estructura del contenido.
- Tipo de preguntas.
- Modelo de embeddings.
- Modelo generativo.
- Tamaño del contexto.
- Requisitos de latencia.
- Resultados de evaluación.

---

9. Metadata

La metadata es información adicional asociada con cada documento o chunk.

Ejemplo:

JSON

```
{
  "document_id": "policy-2026-001",
  "chunk_id": "policy-2026-001-section-03",
  "source": "sharepoint",
  "document_title": "Vacation Policy",
  "department": "Human Resources",
  "tenant_id": "company-a",
  "classification": "internal",
  "language": "en",
```

```
  "version": "3.2",
  "created_at": "2026-01-10",
  "updated_at": "2026-08-20",
  "access_groups": [
    "hr",
    "managers"
  ]
}
```

9.1 ¿Para qué sirve la metadata?

La metadata permite:

- Filtrar resultados.
- Aplicar seguridad.
- Separar tenants.
- Buscar por departamento.
- Identificar documentos recientes.
- Filtrar por idioma.
- Filtrar por tipo de documento.
- Implementar control de versiones.
- Generar citas.
- Auditar el origen de la información.

9.2 Metadata filtering

En lugar de buscar en todos los documentos, el sistema puede aplicar filtros antes o durante la búsqueda.

Ejemplo:

```
None
Buscar documentos sobre vacaciones
donde:

tenant_id = "company-a"
department = "hr"
classification = "internal"
user_has_access = true
```

La metadata no sustituye un sistema de autorización robusto. Los filtros deben estar respaldados por controles de seguridad confiables.

---

10. Embeddings

10.1 ¿Qué es un embedding?

Un embedding es una representación numérica de un texto.

Un texto se transforma en un vector de números que intenta representar su significado semántico.

Ejemplo conceptual:

```
None
"How many vacation days do employees receive?"
                 |
                 v
       [0.12, -0.44, 0.81, ...]
```

El vector real puede tener cientos o miles de dimensiones, dependiendo del modelo utilizado.

10.2 ¿Por qué se utilizan?

Los embeddings permiten comparar textos por similitud semántica.

Por ejemplo, estas preguntas son diferentes en palabras, pero similares en significado:

- “How many vacation days do I have?”
- “What is the annual leave allowance?”
- “How much paid time off is available?”

Un sistema basado solamente en coincidencia exacta de palabras podría no relacionarlas correctamente.

Los embeddings permiten buscar contenido conceptualmente similar.

10.3 Embeddings para documentos y consultas

Durante la ingestión:

```
None
Documento o chunk
        |
        v
Embedding model
        |
        v
Vector almacenado
```

Durante la consulta:

```
None
Pregunta del usuario
        |
        v
Mismo embedding model
        |
        v
Vector de consulta
```

Es importante utilizar modelos compatibles para generar los embeddings de los documentos y de las consultas.

---

11. Búsqueda vectorial

La búsqueda vectorial compara el vector de la consulta con los vectores almacenados.

El objetivo es encontrar los fragmentos más similares semánticamente.

11.1 Similaridad coseno

La similitud coseno mide el ángulo entre dos vectores.

Conceptualmente:

- Valor cercano a 1: vectores con orientación similar.
- Valor cercano a 0: poca relación.
- Valor negativo: orientación opuesta, dependiendo del espacio y del modelo.

La fórmula es:

```
None
cosine_similarity(A, B) =
(A · B) / (||A|| ||B||)
```

Donde:

- `A · B` es el producto punto.
- `||A||` y `||B||` son las magnitudes de los vectores.

11.2 Producto punto

El producto punto también puede utilizarse como métrica de similitud.

Su interpretación depende de cómo fueron entrenados y normalizados los embeddings.

11.3 Distancia euclidiana

Otra opción es medir la distancia entre dos vectores.

En este caso:

- Menor distancia significa mayor similitud.
- Mayor distancia significa menor similitud.

11.4 ANN: Approximate Nearest Neighbor

En una base de datos con millones de vectores, comparar la consulta contra todos los vectores puede ser demasiado lento.

ANN permite encontrar vecinos aproximados de manera eficiente.

Una estructura conocida es:

- HNSW: Hierarchical Navigable Small World.

ANN mejora la velocidad, aunque puede introducir un pequeño compromiso entre precisión y rendimiento.

---

12. Bases de datos vectoriales

Una base de datos vectorial permite almacenar y consultar embeddings.

Ejemplos de tecnologías que pueden utilizarse para búsqueda vectorial:

- Azure AI Search.
- PostgreSQL con pgvector.
- Elasticsearch.
- OpenSearch.
- Pinecone.
- Weaviate.
- Milvus.
- Qdrant.
- Servicios administrados de búsqueda en la nube.

La elección depende de:

- Volumen de datos.
- Requisitos de latencia.
- Integración con la nube.
- Soporte de filtros.
- Alta disponibilidad.
- Seguridad.
- Costos.
- Capacidades híbridas.
- Operación y mantenimiento.
- Requisitos de residencia de datos.

---

13. Búsqueda híbrida

La búsqueda híbrida combina:

1. Búsqueda léxica o keyword search.
2. Búsqueda semántica o vectorial.

13.1 Keyword search

Busca coincidencias de palabras o términos.

Es útil para:

- Nombres exactos.
- IDs.
- Códigos.
- Números de producto.
- Nombres legales.
- Versiones.
- Términos técnicos específicos.

13.2 Vector search

Busca similitud semántica.

Es útil cuando:

- El usuario utiliza sinónimos.
- La pregunta está redactada de forma diferente.
- El documento utiliza terminología distinta.
- Se busca una idea en lugar de una palabra exacta.

13.3 Ventaja de la búsqueda híbrida

La búsqueda híbrida puede mejorar la recuperación porque combina:

- Precisión de términos exactos.
- Flexibilidad semántica.

Ejemplo:

```
None
Consulta:
"Error code A-1042 in the payment service"
```

La búsqueda por palabras puede ser muy importante para encontrar exactamente `A-1042`, mientras que la búsqueda vectorial puede encontrar documentación relacionada con el problema.

---

14. Top-K Retrieval

Top-K representa la cantidad de resultados que el sistema recupera.

Ejemplo:

```
None
Consulta del usuario
       |
       v
Recuperar los 20 chunks más relevantes
       |
       v
```

```
Aplicar filtros o reranking
        |
        v
Enviar los 5 mejores al LLM
```

Es importante diferenciar:

- `K` inicial de recuperación.
- Número de resultados después de reranking.
- Número final de chunks enviados al modelo.

Un valor como `K=5` o `K=10` no es universalmente correcto. Debe ajustarse mediante evaluación.

Recuperar demasiados fragmentos puede:

- Aumentar costos.
- Incrementar latencia.
- Introducir información irrelevante.
- Confundir al modelo.
- Exceder el context window.

Recuperar muy pocos puede:

- Omitir información necesaria.
- Producir respuestas incompletas.
- Perder evidencia importante.

---

15. Reranking

15.1 ¿Qué es un reranker?

Un reranker es un componente que vuelve a evaluar los resultados recuperados y los ordena según su relevancia para la consulta.

El flujo puede ser:

```
None
Consulta
   |
```

```
   v
Recuperación inicial: 50 chunks
   |
   v
Reranker
   |
   v
Top 5 chunks
```

15.2 ¿Por qué es útil?

La búsqueda vectorial puede recuperar fragmentos semánticamente relacionados, pero no necesariamente los más útiles para la pregunta específica.

El reranker analiza con mayor detalle la relación entre:

- Consulta.
- Documento o chunk.

15.3 Diferencia entre búsqueda y reranking

Componente	Función
Vector search	Recupera candidatos rápidamente
Keyword search	Encuentra coincidencias léxicas
Reranker	Ordena candidatos por relevancia más precisa

15.4 Consideración de costos

Un reranker puede mejorar la calidad, pero agrega:

- Latencia.
- Costos de procesamiento.
- Complejidad operativa.

Por eso suele aplicarse a un conjunto limitado de candidatos, no a toda la base de documentos.

---

16. Query Transformation

La pregunta original del usuario no siempre es la mejor consulta para recuperar información.

Por eso, el sistema puede transformar la consulta antes de realizar la búsqueda.

16.1 Query rewriting

El sistema reformula la pregunta.

Ejemplo:

```
None
Pregunta original:
"¿Y cuántos días son para alguien nuevo?"

Consulta reescrita:
"vacation policy annual leave entitlement for new employees"
```

Esto es especialmente útil cuando la pregunta depende de conversaciones anteriores.

16.2 Multi-query retrieval

El sistema genera varias consultas relacionadas.

Ejemplo:

```
None
Consulta original:
"¿Cómo se reporta un incidente crítico?"

Consultas derivadas:
1. Major incident reporting procedure
2. Critical incident escalation process
3. Incident management notification workflow
```

Los resultados se combinan y posteriormente se eliminan duplicados o se vuelven a ordenar.

16.3 HyDE

HyDE significa Hypothetical Document Embeddings.

El sistema genera una respuesta hipotética o documento hipotético y utiliza su embedding para buscar documentos reales similares.

Debe utilizarse con cuidado porque la consulta hipotética no debe tratarse como evidencia verdadera.

---

17. Context Assembly

La recuperación no termina cuando se encuentran los documentos. El sistema debe construir el contexto que será enviado al LLM.

Este proceso se conoce como **context assembly**.

Normalmente incluye:

1. Selección de chunks.
2. Eliminación de duplicados.
3. Ordenamiento por relevancia.
4. Inclusión de títulos y encabezados.
5. Inclusión de metadata útil.
6. Inclusión de referencias.
7. Aplicación de límites de tokens.
8. Construcción del prompt final.

17.1 Ejemplo de contexto

```
None
Source: Employee Vacation Policy
Section: Annual Leave
Document Version: 3.2

Employees are entitled to 20 paid vacation days per calendar
year.
New employees receive a prorated entitlement based on their start
date.
```

Después, este contexto se inserta en el prompt del modelo.

---

18. Prompt para RAG

Un prompt RAG normalmente incluye:

1. Instrucciones del sistema.
2. Pregunta del usuario.
3. Contexto recuperado.
4. Reglas de respuesta.
5. Reglas de citación.
6. Restricciones de seguridad.

Ejemplo:

```
None
System instructions:

You are an enterprise knowledge assistant.

Use only the information provided in the context.
If the context does not contain enough information, say that
the answer cannot be determined from the available sources.

Do not invent policies, dates, names, or procedures.

Context:

[Document 1]
Employees are entitled to 20 paid vacation days per year.

[Document 2]
New employees receive a prorated entitlement.

User question:

How many vacation days does a new employee receive?

Response requirements:

- Answer in English.
- Explain the prorated rule.
- Include the source references.
- Do not infer an exact number without the employee's start date.
```

---

19. Generation

En esta etapa, el LLM recibe:

- La pregunta.
- Las instrucciones.
- El contexto recuperado.
- Las restricciones de respuesta.

El modelo genera una respuesta utilizando esa información.

Es importante recordar que el LLM no “consulta directamente” la base vectorial. Generalmente, la aplicación realiza la búsqueda y luego envía los resultados al modelo como parte del prompt.

```
None
Vector database
      |
      v
Application / RAG orchestration
      |
      v
Prompt with context
      |
      v
LLM
```

---

20. Grounding

Grounding significa fundamentar la respuesta en información externa proporcionada al modelo.

En RAG, el grounding busca que la respuesta:

- Se base en documentos recuperados.
- Sea consistente con las fuentes.
- Evite inventar información.
- Pueda ser verificada.

Sin embargo, incluir documentos en el prompt no garantiza que el modelo los utilice correctamente.

Por eso se necesitan:

- Instrucciones claras.
- Evaluaciones.
- Validación de citas.
- Controles de respuesta.
- Reglas para indicar cuando no hay suficiente información.

---

21. Citas y trazabilidad

Un sistema empresarial debería conservar la relación entre la respuesta y las fuentes utilizadas.

Una cita puede incluir:

- Nombre del documento.
- URL o ubicación.
- ID del documento.
- Página.
- Sección.
- Versión.
- Fragmento utilizado.
- Fecha de actualización.

Ejemplo:

```
None
Source:
Employee Vacation Policy, version 3.2, section "Annual Leave"
```

21.1 Citas generadas vs citas verificadas

Es importante distinguir:

Cita generada por el LLM

El modelo escribe una referencia basándose en el contexto recibido.

Cita verificada por la aplicación

La aplicación controla que la cita corresponda realmente a un documento y fragmento recuperado.

En sistemas empresariales, es preferible que las citas sean generadas a partir de identificadores controlados por la aplicación y no inventadas libremente por el modelo.

---

22. Flujo RAG completo

None

```
                    FLUJO DE INGESTIÓN

Fuentes de datos
      |
      v
Conectores / Crawlers
      |
      v
Extracción y parsing
      |
      v
Limpieza y normalización
      |
      v
Chunking
      |
      v
Metadata enrichment
      |
      v
Embedding generation
      |
      v
Vector / Search Index
```

None

```
                    FLUJO DE CONSULTA

Usuario
   |
   v
Pregunta
   |
   v
```

```
Authentication / Authorization
   |
   v
Query processing
   |
   v
Embedding de la consulta
   |
   v
Hybrid / Vector Search
   |
   v
Metadata filters
   |
   v
Reranking
   |
   v
Context assembly
   |
   v
Prompt construction
   |
   v
LLM
   |
   v
Respuesta
   |
   v
Citas / Validaciones / Logging
```

---

23. RAG y seguridad empresarial

La seguridad es uno de los aspectos más importantes en una plataforma RAG.

23.1 Problema principal

Un usuario no debería recibir información a la que no tiene acceso solamente porque el sistema la recuperó.

Ejemplo:

- Un empleado de Finanzas pregunta por una política.
- El sistema recupera un documento confidencial de Recursos Humanos.
- El LLM genera una respuesta utilizando ese documento.

Esto representa una violación de seguridad.

23.2 Controles necesarios

- Autenticación.
- Autorización.
- RBAC.
- ABAC.
- Filtros por tenant.
- Filtros por grupo o usuario.
- Seguridad a nivel de documento.
- Seguridad a nivel de chunk.
- Control de acceso en el origen.
- Validación de permisos durante la consulta.
- Auditoría.
- Cifrado en tránsito.
- Cifrado en reposo.
- Gestión segura de secretos.

23.3 Security trimming

Security trimming significa que los resultados se limitan según los permisos del usuario.

Ejemplo:

```
None
User:
department = Finance
groups = ["finance-users"]

Search filter:
tenant_id = current_tenant
access_groups contains "finance-users"
```

El filtro debe basarse en permisos reales y no solamente en un valor enviado por el cliente.

23.4 No confiar en el prompt para la seguridad

Una instrucción como:

```
None
Do not reveal confidential information.
```

no sustituye los controles de autorización.

La seguridad debe aplicarse antes de que los documentos lleguen al contexto del LLM.

---

24. Multi-tenancy en RAG

En una plataforma empresarial, varios clientes, departamentos o equipos pueden utilizar el mismo sistema.

Esto introduce el problema de aislamiento de datos.

24.1 Estrategias de aislamiento

Índice separado por tenant

Cada tenant tiene su propio índice.

Ventajas:

- Aislamiento claro.
- Menor riesgo de mezcla de datos.
- Administración independiente.

Desventajas:

- Más infraestructura.
- Mayor costo operativo.
- Más complejidad para administrar muchos tenants.

Índice compartido con filtros

Todos los documentos están en un índice, pero cada registro incluye `tenant_id`.

Ventajas:

- Menor costo.

- Administración centralizada.
- Escalabilidad operativa.

Desventajas:

- Requiere filtros obligatorios.
- Un error de implementación puede causar fuga de información.
- La seguridad debe probarse cuidadosamente.

Base de datos separada por tenant

Cada tenant tiene su propia base de datos o esquema.

Puede ofrecer un nivel de aislamiento intermedio o alto, dependiendo de la implementación.

24.2 Recomendación conceptual

La estrategia debe seleccionarse según:

- Sensibilidad de los datos.
- Requisitos regulatorios.
- Cantidad de tenants.
- Costos.
- Requisitos de residencia de datos.
- Riesgo de fuga de información.
- Necesidades de personalización.
- Requisitos de disponibilidad.

---

25. RAG y observabilidad

Un sistema RAG requiere más observabilidad que una API tradicional porque la respuesta depende de múltiples etapas.

25.1 Métricas de ingestión

- Número de documentos procesados.
- Número de chunks creados.
- Documentos fallidos.
- Tiempo de procesamiento.
- Cantidad de tokens procesados.
- Errores de extracción.
- Tiempo desde la actualización del documento hasta su indexación.
- Número de documentos eliminados o desactualizados.

25.2 Métricas de recuperación

- Tiempo de búsqueda.
- Número de resultados recuperados.
- Score de similitud.
- Score del reranker.
- Tasa de consultas sin resultados.
- Tasa de resultados irrelevantes.
- Filtros aplicados.
- Índice consultado.
- Versión del embedding model.

25.3 Métricas de generación

- Latencia del LLM.
- Input tokens.
- Output tokens.
- Costo estimado.
- Errores del proveedor.
- Time to first token.
- Tiempo total de respuesta.
- Modelo utilizado.
- Versión del prompt.
- Número de reintentos.

25.4 Trazabilidad distribuida

Una consulta puede generar múltiples llamadas:

```
None
API Gateway
   |
   v
RAG Orchestrator
   |
   +--> Embedding service
   |
   +--> Search service
   |
   +--> Reranker
   |
   +--> LLM provider
   |
```

```
   v
Response formatter
```

Es recomendable utilizar un `trace_id` para relacionar todas las operaciones.

---

26. Evaluación de RAG

Una aplicación RAG debe evaluarse sistemáticamente.

No es suficiente preguntar manualmente si “parece responder bien”.

La evaluación debe analizar al menos dos áreas:

1. Calidad de recuperación.
2. Calidad de generación.

26.1 Evaluación de retrieval

Busca determinar si el sistema recupera la información correcta.

Preguntas:

- ¿El documento correcto fue recuperado?
- ¿El fragmento relevante aparece entre los primeros resultados?
- ¿Los resultados contienen suficiente información?
- ¿Se recuperó información irrelevante?
- ¿La búsqueda respeta los filtros de seguridad?

Métricas comunes:

- Precision@K.
- Recall@K.
- Hit Rate.
- Mean Reciprocal Rank.
- NDCG.

Recall@K

Mide si la información relevante aparece dentro de los primeros `K` resultados.

Precision@K

Mide qué proporción de los primeros K resultados es relevante.

26.2 Evaluación de generación

Busca determinar si la respuesta final es correcta y útil.

Aspectos:

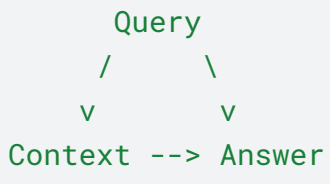
- Correctness.
- Relevance.
- Completeness.
- Fluency.
- Groundedness.
- Faithfulness.
- Citation accuracy.
- Refusal quality.

26.3 RAG Triad

La denominada RAG Triad suele analizar tres relaciones:

1. Query → Context
   ¿El contexto recuperado es relevante para la pregunta?
2. Context → Answer
   ¿La respuesta está respaldada por el contexto?
3. Query → Answer
   ¿La respuesta realmente responde a la pregunta?

```
None
```



26.4 Dataset de evaluación

Un dataset de evaluación puede contener:

```JSON
{
  "question": "How many vacation days are provided?",
  "expected_answer": "Employees receive 20 days per year.",
  "expected_sources": [
    "vacation-policy-v3.2"
  ],
  "tenant_id": "company-a"
}
```

El dataset debe incluir preguntas variadas:

- Preguntas sencillas.
- Preguntas ambiguas.
- Preguntas sin respuesta.
- Preguntas que requieren varios documentos.
- Preguntas con sinónimos.
- Preguntas en distintos idiomas.
- Preguntas que intentan acceder a información restringida.

---

27. Fallos comunes en sistemas RAG

27.1 Recuperación incorrecta

El sistema no encuentra el documento relevante.

Posibles causas:

- Chunking deficiente.
- Embeddings inadecuados.
- Metadata incorrecta.
- Consulta mal formulada.
- Índice desactualizado.
- Falta de búsqueda híbrida.
- K demasiado bajo.

27.2 Contexto insuficiente

El documento correcto fue encontrado, pero el fragmento no contiene suficiente información.

Posibles soluciones:

- Aumentar el tamaño de los chunks.
- Usar overlap.
- Recuperar chunks vecinos.
- Utilizar chunking jerárquico.
- Recuperar la sección completa.
- Incorporar contexto del documento padre.

27.3 Contexto irrelevante

Se recuperan demasiados fragmentos que no ayudan a responder.

Posibles soluciones:

- Reranking.
- Mejores filtros.
- Menor K.
- Query rewriting.
- Mejorar embeddings.
- Ajustar chunking.

27.4 Respuesta no fundamentada

El LLM responde utilizando información que no aparece en el contexto.

Posibles soluciones:

- Instrucciones de grounding.
- Respuesta “no encontrado”.
- Validación de claims.
- Evaluación de faithfulness.
- Citas verificadas.
- Modelos o prompts más adecuados.

27.5 Documentos desactualizados

El sistema responde con una política antigua.

Posibles soluciones:

- Versionado.
- Fechas de actualización.
- Eliminación de documentos obsoletos.
- Filtros por vigencia.
- Procesamiento incremental.
- Reindexación controlada.

27.6 Fuga de información

El sistema recupera documentos que el usuario no debería ver.

Posibles soluciones:

- Security trimming.
- Validación de permisos.
- Filtros obligatorios.
- Aislamiento por tenant.
- Pruebas de autorización.
- Auditoría.

27.7 Context window excedido

Se recupera demasiado contenido y el prompt supera el límite del modelo.

Posibles soluciones:

- Reducir el número de chunks.
- Aplicar reranking.
- Resumir documentos.
- Utilizar compresión de contexto.
- Seleccionar solamente evidencia relevante.
- Utilizar un modelo con mayor ventana de contexto.

---

28. RAG avanzado

28.1 Parent-child retrieval

Se indexan fragmentos pequeños, pero cada fragmento conserva una referencia a una sección o documento padre.

El sistema recupera el fragmento preciso y después puede incluir contexto adicional del padre.

28.2 Graph RAG

Graph RAG utiliza grafos de conocimiento para representar relaciones entre:

- Entidades.
- Documentos.
- Conceptos.

- Personas.
- Productos.
- Eventos.

Puede ser útil para preguntas que requieren conectar múltiples entidades o relaciones.

28.3 Agentic RAG

En Agentic RAG, un agente decide:

- Qué fuente consultar.
- Qué consulta realizar.
- Si necesita otra búsqueda.
- Si debe usar una API.
- Si necesita combinar resultados.
- Si debe verificar la respuesta.

Esto aumenta la flexibilidad, pero también la complejidad, la latencia y los riesgos operativos.

28.4 Multimodal RAG

Multimodal RAG trabaja con información como:

- Texto.
- Imágenes.
- Tablas.
- Diagramas.
- Audio.
- Video.

Por ejemplo, un sistema puede recuperar una tabla financiera o una imagen de arquitectura además de texto.

28.5 Structured RAG

Structured RAG utiliza fuentes estructuradas, como:

- SQL.
- APIs.
- JSON.
- Sistemas transaccionales.

No todas las preguntas deben resolverse con búsqueda vectorial.

Ejemplo:

“¿Cuántos tickets críticos se abrieron ayer?”

Esta pregunta probablemente requiere consultar una base de datos o una API, no buscar documentos semánticamente.

---

29. Cuándo no utilizar RAG

RAG no es siempre la mejor solución.

Puede no ser necesario cuando:

- La información está completamente contenida en el prompt.
- La tarea es principalmente de transformación de texto.
- Se requiere una operación transaccional exacta.
- La respuesta debe calcularse con datos estructurados.
- Se necesita una consulta SQL precisa.
- El problema es principalmente cambiar el estilo o comportamiento del modelo.
- No existen fuentes confiables para recuperar.

En algunos casos es mejor utilizar:

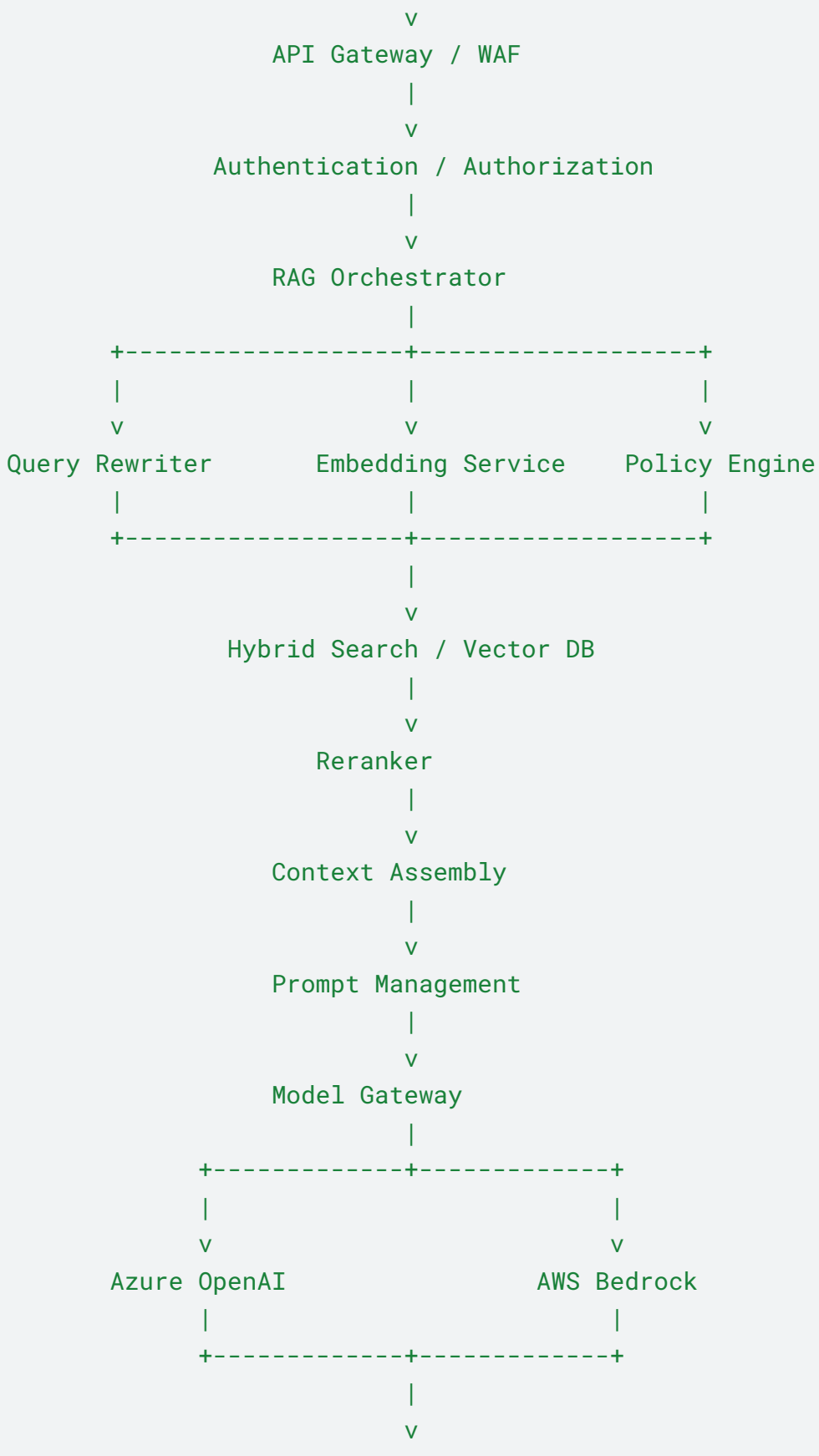
- APIs.
- SQL.
- Búsqueda tradicional.
- Fine-tuning.
- Function calling.
- Sistemas expertos.
- Reglas deterministas.
- Modelos especializados.

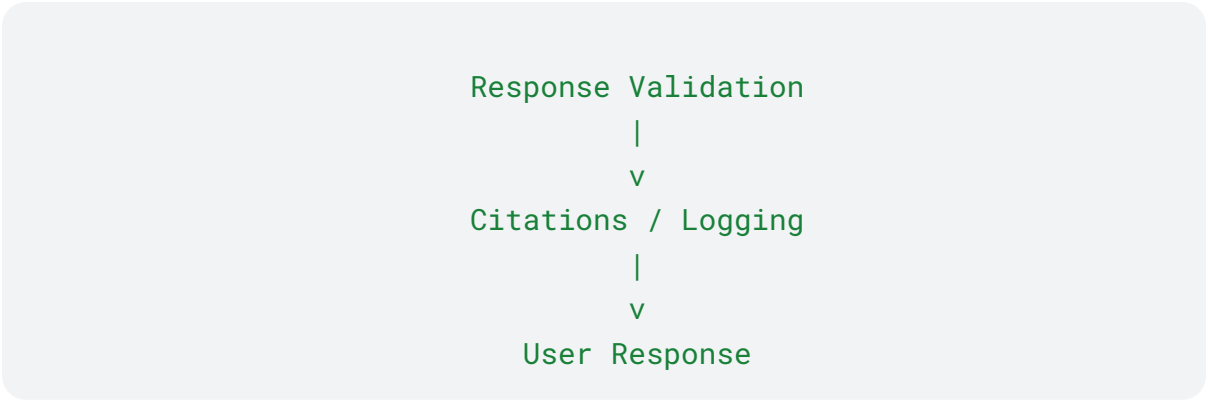
---

30. Arquitectura empresarial de referencia

Una arquitectura empresarial podría verse así:

```
None
                    CLIENT APPLICATIONS
              Web App | Mobile | Internal Tools
                                |
```


API Gateway / WAF
Authentication / Authorization
RAG Orchestrator
Query Rewriter
Embedding Service
Policy Engine
Hybrid Search / Vector DB
Reranker
Context Assembly
Prompt Management
Model Gateway
Azure OpenAI
AWS Bedrock



Componentes transversales

- Secrets management.
- Monitoring.
- Distributed tracing.
- Cost tracking.
- Evaluation pipelines.
- Prompt versioning.
- CI/CD.
- Data governance.
- Audit logging.
- Rate limiting.
- Quotas.
- Incident management.
- Disaster recovery.
- Multi-region strategy.

---

31. Ejemplo empresarial: asistente de soporte

Supongamos que una empresa quiere crear un asistente para responder preguntas sobre incidentes y procedimientos operativos.

Fuentes

- Manuales de operación.
- Historial de incidentes.
- Runbooks.
- Base de conocimiento.
- Documentación de ServiceNow.
- Procedimientos de escalamiento.

- Políticas de Major Incident Management.

Flujo

1. Se extraen documentos y tickets autorizados.
2. Se limpian y dividen en chunks.
3. Se agregan metadatos como servicio, equipo, severidad y fecha.
4. Se generan embeddings.
5. Se indexan los fragmentos.
6. El usuario pregunta:
   “¿Qué procedimiento debemos seguir cuando un servicio crítico presenta una interrupción?”
7. El sistema recupera runbooks y políticas relevantes.
8. Se aplican filtros por equipo y permisos.
9. Un reranker selecciona los fragmentos más relevantes.
10. El LLM genera una respuesta.
11. La aplicación incluye las fuentes.
12. La consulta se registra para evaluación y auditoría.

Posible respuesta

La respuesta podría incluir:

- Pasos iniciales.
- Equipo responsable.
- Criterios de escalamiento.
- Requisitos de comunicación.
- Referencias al runbook.
- Advertencia si falta información.

---

32. Relación con mi experiencia profesional

Mi experiencia en APIs, cloud, MLOps, pipelines y sistemas empresariales se relaciona directamente con la construcción de plataformas RAG.

Experiencia transferible

APIs y microservicios

RAG normalmente requiere servicios para:

- Ingestión.
- Búsqueda.
- Consulta.
- Generación.
- Administración de prompts.
- Evaluación.
- Métricas.

La experiencia con FastAPI, REST, Pydantic y OpenAPI es relevante.

Cloud

Una plataforma RAG puede utilizar:

- Object storage.
- Servicios administrados de búsqueda.
- Model endpoints.
- Kubernetes.
- Serverless.
- Secret management.
- Monitoring.
- Networking privado.

La experiencia con AWS y Azure es aplicable.

MLOps

RAG necesita prácticas de MLOps y LLMOps para:

- Versionar modelos de embeddings.
- Versionar prompts.
- Automatizar evaluaciones.
- Desplegar servicios.
- Monitorear calidad.
- Controlar costos.
- Gestionar cambios.
- Reproducir resultados.

Pipelines

La ingestión de documentos es un pipeline de datos y AI que puede incluir:

- Extracción.
- Transformación.
- Validación.
- Indexación.
- Reprocesamiento.
- Manejo de errores.
- Observabilidad.

ServiceNow e incidentes

La experiencia con incidentes, análisis histórico y predicción de soluciones puede relacionarse con RAG para:

- Recuperar incidentes similares.
- Buscar procedimientos de resolución.
- Identificar equipos responsables.
- Enriquecer tickets.
- Sugerir runbooks.
- Proporcionar evidencia histórica.

Forma de explicarlo en entrevista

> "I see RAG as a combination of data engineering, search, API engineering, and LLM orchestration. My background in cloud architecture, MLOps, pipelines, and enterprise APIs is directly relevant because a production RAG system is not just a prompt connected to a model. It requires reliable ingestion, indexing, security, observability, evaluation, and operational controls."

---

33. Preguntas de entrevista

Conceptos generales

1. What is RAG and why is it useful?
2. How is RAG different from fine-tuning?
3. What are the main components of a RAG architecture?
4. What is the difference between ingestion and query-time retrieval?
5. What are embeddings?
6. What is a vector database?
7. What is hybrid search?
8. What is a reranker?
9. What is context assembly?
10. What is grounding?

Diseño y arquitectura

11. How would you design a multi-tenant RAG platform?
12. How would you prevent one tenant from accessing another tenant's documents?
13. How would you handle document updates and deletions?
14. How would you choose between a shared index and separate indexes?
15. How would you reduce RAG latency?

16. How would you control LLM and embedding costs?
17. How would you support multiple model providers?
18. How would you implement prompt versioning?
19. How would you monitor a RAG pipeline?
20. How would you design disaster recovery for a RAG platform?

Calidad

21. How do you evaluate retrieval quality?
22. How do you evaluate answer faithfulness?
23. What would you do if the correct document is not retrieved?
24. How would you detect hallucinations?
25. How would you validate citations?
26. What is the RAG Triad?
27. How would you build a golden evaluation dataset?
28. How would you evaluate a system with no-answer questions?

---

34. Respuesta corta para entrevista

> “RAG is an architecture that retrieves relevant information from external sources and provides it to an LLM as context before generating a response. A production-grade RAG system includes ingestion pipelines, document parsing, chunking, embeddings, vector or hybrid search, metadata filtering, reranking, prompt construction, generation, citations, security, observability, and evaluation. The main challenge is not simply connecting a vector database to an LLM, but ensuring that the system retrieves the right information, respects access controls, produces grounded answers, and operates reliably at enterprise scale.”

---

35. Resumen ejecutivo

RAG permite que un LLM utilice información externa para responder preguntas.

Su flujo principal es:

```
None
Fuentes
→ Ingestión
→ Chunking
→ Embeddings
```

```
→ Índice
→ Recuperación
→ Reranking
→ Context Assembly
→ Prompt
→ LLM
→ Respuesta fundamentada
```

Los aspectos más importantes de RAG son:

- Calidad de los documentos.
- Estrategia de chunking.
- Embeddings adecuados.
- Búsqueda vectorial o híbrida.
- Metadata y filtros.
- Reranking.
- Construcción correcta del contexto.
- Seguridad y aislamiento de datos.
- Evaluación de retrieval y generación.
- Observabilidad.
- Control de costos.
- Trazabilidad y citas.

La idea fundamental es:

> RAG no consiste solamente en hacer una búsqueda antes de llamar a un LLM.
> Es una arquitectura completa que conecta datos empresariales, búsqueda, seguridad, generación y operación confiable.

---

36. Temas opcionales para profundizar

Si se requiere mayor profundidad técnica, investigar los siguientes temas:

Embeddings

None

```
Explain embeddings for RAG systems, including vector spaces,
semantic similarity, cosine similarity, dot product,
normalization, embedding dimensions, and how embedding models are
evaluated.
```

Chunking

None

```
Explain advanced chunking strategies for enterprise RAG systems, including fixed-size chunking, recursive chunking, semantic chunking, hierarchical chunking, parent-child retrieval, overlap, and evaluation methods.
```

Vector Search

None

```
Explain vector search in enterprise RAG systems, including ANN, HNSW, IVF, cosine similarity, dot product, filtering, indexing, recall, latency, and scalability.
```

Hybrid Search

None

```
Explain hybrid search for RAG systems, comparing keyword search, BM25, vector search, reciprocal rank fusion, and reranking.
```

Reranking

None

```
Explain reranking in RAG pipelines, including cross-encoders, bi-encoders, candidate retrieval, relevance scoring, latency, and production trade-offs.
```

RAG Evaluation

None

```
Explain how to evaluate enterprise RAG systems, including retrieval precision, recall, hit rate, MRR, NDCG, faithfulness, groundedness, answer relevance, citation accuracy, and the RAG Triad.
```

Enterprise Security

None

```
Explain security architecture for multi-tenant enterprise RAG systems, including RBAC, ABAC, security trimming, document-level permissions, tenant isolation, prompt injection, data leakage, encryption, and auditability.
```

Advanced RAG

None

```
Explain advanced RAG architectures, including Graph RAG, Agentic RAG, multimodal RAG, query rewriting, multi-query retrieval, HyDE, contextual compression, and parent-child retrieval.
```